

Proof Search Beyond Term Rewriting

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

Tactic-based proof assistants—LEAN 4, COQ/Ltac2, Meta-F*—decompose proof search into composable steps that rewrite proof *terms*. Yet none of these frameworks can manipulate trust annotations, negotiate overlap treaties between modular proof fragments, or generate repair frontiers for obstructed gluing. We introduce *semantic moves*: first-class, composable proof-search operations that act on the full geometric proof state—coordinates, covers, overlaps, evidence bundles, trust levels, and obstructions. We define 13 semantic move kinds, organized into a typed morphism category over proof states, and prove that every move preserves judgment well-formedness. We map the 13 formal kinds to 10 named moves—VERIFY, CONSTRUCT, NORMALIZE, GLUE, RESTRICT, TRANSPORT, REPAIR, PROMOTE, CHALLENGE, and ESCALATE—that form the orchestration vocabulary of the JUGE system. Seven of the 13 kinds are geometric and have no analogue in existing tactic frameworks. Evaluation on 100 Python programs across four categories (pure function, multi-function, class, module with dependencies) shows that the geometric fraction rises from 19.6% for pure functions to 41.8% for modules, and that morphisms exceed coordinates for 76% of multi-function programs, confirming that richer program structure demands the geometric apparatus.

Keywords

semantic moves, proof search, tactic frameworks, judgment geometry, Grothendieck topology, proof-state category, trust algebra

1 Introduction

Modern proof assistants organize proof search around *tactics*: small programs that transform a proof goal into subgoals. In LEAN 4, the tactic monad threads a `MetavarContext` through combinators such as `apply`, `simp`, and `ring` [1, 2]. Coq’s Ltac2 provides a typed meta-language over the proof state [3]. Meta-F* extends the weakest-precondition discipline with user-defined strategies that discharge verification conditions via SMT [4].

All three systems share a common assumption: the proof state is a *term*—a lambda-expression, a verification condition, or a metavariable context—and every tactic step rewrites that term. This “term-rewriting” paradigm suffices for single-site reasoning but leaves three capabilities on the table:

- (a) **Trust manipulation.** No tactic in LEAN 4 or COQ can raise or lower the confidence level of a proof obligation.
- (b) **Overlap negotiation.** When two modules share an interface, existing frameworks offer no first-class operation to glue local proofs along the overlap or to record a cohomological obstruction.
- (c) **Repair frontiers.** When gluing fails, no existing tactic can refine the covering family to eliminate the obstruction.

1.1 Contributions

1. We define a *proof-state category* **PS** whose objects are well-formed proof states and whose morphisms are semantic moves (Section 3).
2. We introduce 13 move kinds—6 classical and 7 geometric—and prove soundness and functoriality (Section 5).
3. We map the 13 formal kinds to 10 named moves in the JU_{GEO} orchestration layer (Section 6).
4. We prove that the 7 geometric moves cannot be simulated by any finite combination of LEAN 4, Ltac2, or Meta-F^{*} tactics (Section 12).
5. We describe the deep JU_{GEO} API for moves, coordinates, morphisms, and descent (Section 10).
6. We evaluate on 100 programs across four categories, showing that morphisms exceed coordinates for multi-function programs and that the geometric fraction reaches 41.8% (Section 11).

2 Background

2.1 Proof search in Lean 4

LEAN 4’s tactic monad [1] threads a `MetavarContext` through tactic combinators. The state contains metavariables (proof obligations), a local context, and substitutions. Tactics transform this state by unification, rewriting, or invoking decision procedures.

2.2 Ltac2 in Coq

Ltac2 [3] provides a typed meta-language for Coq. Its state is a proof term with holes; tactics fill holes by constructing sub-terms.

2.3 Meta-F^{*} strategies

Meta-F^{*} [4] adds SMT integration to the tactic framework, but the state remains a verification condition—a term in the weakest-precondition calculus.

2.4 Judgment geometry recap

The JU_{GEO} framework (Papers 1–5) organizes verification around semantic sites, judgment sheaves, a trust algebra, and descent. The proof state includes coordinates, covers, overlaps, evidence bundles, trust levels, and obstructions—far richer than a proof term.

3 The Semantic Move

3.1 Proof state

Definition 3.1 (Proof state). A *proof state* is a 7-tuple $\text{PS} = (\mathcal{S}, \mathcal{J}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ where $\mathcal{S}$ is the semantic site, $\mathcal{J}$ the judgment sheaf, $\mathcal{E}$ the evidence bundle, $\mathcal{O}$ the obligation set, $\mathcal{B}$ the obstruction presheaf, $\mathcal{T}$ the trust annotation, and Π the provenance log. A proof state is *well-formed* if it satisfies:

- (WF1) every judgment is supported by evidence;

- (WF2) trust levels are consistent with evidence quality;
- (WF3) the obstruction presheaf is compatible with the Grothendieck topology.

3.2 Proof-state category

Definition 3.2 (Proof-state category). The *proof-state category* $\mathbf{PS}$ has:

- Objects: well-formed proof states.
- Morphisms: semantic moves $m : \mathbf{PS}_1 \rightarrow \mathbf{PS}_2$.
- Identity: the no-op move $\text{id}_{\mathbf{PS}}$.
- Composition: sequential application of moves.

3.3 Semantic move

Definition 3.3 (Semantic move). A *semantic move* is a 7-tuple $m = (\text{kind}, \text{tgt}, \text{pre}, \text{post}, \text{gain}, \text{cost}, \text{floor})$ where $\text{kind} \in \text{MoveKind}$ is the move kind, tgt is the target coordinate, pre and post are pre- and post-conditions on the proof state, gain is the expected progress metric, cost is the estimated computational cost, and floor is the minimum trust level required.

3.4 Composition

Definition 3.4 (Move composition). Sequential composition: $(m_2 \circ m_1)(\mathbf{PS}) = m_2(m_1(\mathbf{PS}))$. Parallel composition: $(m_1 \otimes m_2)(\mathbf{PS}) = m_1(\mathbf{PS}|_{c_1}) \cup m_2(\mathbf{PS}|_{c_2})$ for disjoint coordinates c_1, c_2 .

3.5 Soundness

Theorem 3.5 (Move soundness). *Every semantic move preserves well-formedness: if $\mathbf{PS}$ is well-formed and $m : \mathbf{PS} \rightarrow \mathbf{PS}'$ is any semantic move, then $\mathbf{PS}'$ is well-formed.*

Proof. We verify WF1–WF3 for each of the 13 move kinds. The proof proceeds by case analysis on $\text{kind}(m) \in \text{MoveKind}$.

Classical moves (6 kinds).

compose. A compose move builds a new evidence term $e' = e_2 \circ e_1$ from two existing evidence terms $e_1, e_2 \in \mathcal{E}$. Since e_1 and e_2 each support their respective judgments (WF1 for $\mathbf{PS}$), the composite e' supports the composed judgment. Trust is set to $\mathcal{T}(e') = \mathcal{T}(e_1) \oplus \mathcal{T}(e_2)$, which is consistent by the join property of the trust algebra (WF2). No obstructions are introduced (WF3 trivial).

split. Splitting a conjunction $\varphi_1 \wedge \varphi_2$ into two sub-obligations preserves evidence support: $e|_{\varphi_i}$ is defined by the projection principle of evidence bundles. Trust is inherited ($\mathcal{T}(e|_{\varphi_i}) = \mathcal{T}(e)$), and no site structure is modified.

strengthen. Strengthening replaces an obligation φ with a logically stronger φ' such that $\varphi' \Rightarrow \varphi$. If e' supports φ' , then e' supports φ a fortiori (WF1). Trust and obstructions are unchanged.

weaken. Weakening replaces φ with a logically weaker φ' . The existing evidence e may no longer support φ' directly, but the move records a pending obligation for φ' , maintaining WF1 by the obligation-tracking invariant.

add_obligation. Adding an obligation o to $\mathcal{O}$ does not affect existing evidence (WF1) and sets $\mathcal{T}(o) = \text{UNVERIFIED}$ (WF2). No obstructions are modified (WF3).

discharge_obligation. Discharging $o \in \mathcal{O}$ with evidence e removes o and adds e to $\mathcal{E}$, preserving WF1. Trust is set by the evidence source (WF2).

Geometric moves (7 kinds).

restrict_to. Restricting the proof state to a sub-coordinate $c' \subseteq c$ yields $\text{PS}|_{c'}$. By the sheaf restriction axiom, evidence restricts faithfully: $\mathcal{E}|_{c'}$ supports $\mathcal{J}|_{c'}$ (WF1). Trust is monotone under restriction: $\mathcal{T}(\mathcal{E}|_{c'}) \preceq \mathcal{T}(\mathcal{E})$ (WF2). The obstruction presheaf restricts compatibly with the induced topology on the sub-site (WF3).

merge. Merging two proof states $\text{PS}|_{c_1}$ and $\text{PS}|_{c_2}$ along an overlap $c_1 \cap c_2$ requires agreement on the overlap. If $\mathcal{E}|_{c_1|_{c_1 \cap c_2}} = \mathcal{E}|_{c_2|_{c_1 \cap c_2}}$ (the compatibility condition), the merged evidence supports the merged judgment (WF1). Trust is the conservative join: $\mathcal{T}_{\text{merged}} = \mathcal{T}(\mathcal{E}|_{c_1}) \oplus \mathcal{T}(\mathcal{E}|_{c_2})$ (WF2). If the compatibility condition fails, the move adds an obstruction to $\mathcal{B}$, which is compatible with the topology by construction (WF3).

transport_along. Transporting along a morphism $f : c_1 \rightarrow c_2$ applies the pushforward functor f_* to evidence. Since f_* is a functor, it preserves the support relation (WF1). Trust is monotone under pushforward by the functoriality of the trust algebra (WF2). The obstruction presheaf transforms covariantly along f (WF3).

add_obstruction. Adding $b \in \mathcal{B}(c)$ records a new obstruction at coordinate c . Existing evidence is unaffected (WF1), trust is unchanged (WF2), and $\mathcal{B}$ remains a valid presheaf because single-point extensions are compatible with any Grothendieck topology (WF3).

resolve_obstruction. Resolving $b \in \mathcal{B}(c)$ removes it and may add evidence. WF1 is maintained because resolution either provides evidence or defers to a sub-obligation. Trust for the new evidence is set by the resolution source (WF2). Removing an element from a presheaf yields a sub-presheaf, which is compatible (WF3).

repair. Repair modifies the covering family $\text{Cov}(c)$ to eliminate an obstruction. The key observation is that refining a cover (replacing $\{U_i\}$ with a finer $\{V_j\}$) preserves existing evidence by the sheaf condition: sections over $\{U_i\}$ restrict to sections over $\{V_j\}$ (WF1). Trust is preserved because restriction is monotone (WF2). The refined topology is still a Grothendieck topology (stability and transitivity axioms are preserved under refinement), so $\mathcal{B}$ remains compatible (WF3).

descend. Descent computes a global section from a compatible family of local sections. The descent datum is a family $\{s_i \in \mathcal{E}(U_i)\}$ satisfying the cocycle condition $s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}$. By the sheaf axiom, there exists a unique global section s restricting to each s_i (WF1). Trust is the meet of local trusts: $\mathcal{T}(s) = \bigwedge_i \mathcal{T}(s_i)$ (WF2). The descent process resolves all obstructions in H^1 associated with the cover, preserving compatibility (WF3). $\square$

We record the three well-formedness invariants as separate lemmas for reference.

Lemma 3.6 (WF1: Evidence support). *For every judgment $j \in \mathcal{J}$ and every coordinate $c \in \text{Ob}(\mathcal{S})$, if j is assigned to c then there exists $e \in \mathcal{E}(c)$ with $e \vdash j$, or j is recorded in $\mathcal{O}$.*

Proof. By induction on move application. The base case holds for the initial proof state, which assigns UNVERIFIED obligations for all judgments. Each move either provides evidence (compose, split, strengthen, discharge_obligation, merge, transport_along, descend) or records an obligation (weaken, add_obligation), maintaining the invariant. $\square$

Lemma 3.7 (WF2: Trust consistency). *For every evidence term $e \in \mathcal{E}(c)$, the trust level $\mathcal{T}(e)$ is consistent with the evidence quality: $\mathcal{T}(e) \preceq \text{ceiling}(\text{source}(e))$.*

Proof. Trust assignments are controlled by the trust algebra. New evidence from a solver has $\mathcal{T}(e) = \text{SOLVER}$, from a runtime test has $\mathcal{T}(e) = \text{RUNTIME}$, and from a formal proof has $\mathcal{T}(e) = \text{PROOF}$. The ceiling function is monotone, and all trust operations ($\oplus$, $\ominus$, $\uparrow_\pi$, $\downarrow_\chi$) respect the ceiling bound by the trust algebra axioms (Paper 4). $\square$

Lemma 3.8 (WF3: Obstruction compatibility). *The obstruction presheaf $\mathcal{B}$ satisfies the presheaf condition: for every morphism $f : c_1 \rightarrow c_2$ in $\mathcal{S}$, the restriction map $\mathcal{B}(f) : \mathcal{B}(c_2) \rightarrow \mathcal{B}(c_1)$ is well-defined and functorial.*

Proof. We verify the presheaf axioms. For identity morphisms, $\mathcal{B}(\text{id}_c) = \text{id}_{\mathcal{B}(c)}$ holds because no obstruction is transformed. For composition, $\mathcal{B}(g \circ f) = \mathcal{B}(f) \circ \mathcal{B}(g)$ holds because restriction of obstructions is functorial (it is defined by restriction of sections in the ambient sheaf). Each move preserves these axioms: adding an obstruction extends $\mathcal{B}$ at a single coordinate (the extension is compatible because the ambient sheaf provides the restriction maps), and resolving an obstruction removes an element (sub-presheaves of presheaves are presheaves). $\square$

3.6 Functoriality

Theorem 3.9 (Functoriality). *Sequential composition is associative and unital. Parallel composition is symmetric monoidal.*

3.7 Move trace

Definition 3.10 (Move trace). The *geometric weight* of a move trace $\sigma = (m_1, \dots, m_n)$ is $w_g(\sigma) = |\{i : \text{kind}(m_i) \in \text{Geometric}\}|/n$.

4 Move Algebra

The proof-state category $\mathbf{PS}$ inherits algebraic structure from the composition and interaction of moves. We develop this algebra, establishing composition laws, idempotency properties, and commutation relations between classical and geometric moves.

4.1 Composition laws

Proposition 4.1 (Associativity of sequential composition). *For moves $m_1 : \mathbf{PS}_1 \rightarrow \mathbf{PS}_2$, $m_2 : \mathbf{PS}_2 \rightarrow \mathbf{PS}_3$, $m_3 : \mathbf{PS}_3 \rightarrow \mathbf{PS}_4$, we have $(m_3 \circ m_2) \circ m_1 = m_3 \circ (m_2 \circ m_1)$.*

Proof. Both sides evaluate to $m_3(m_2(m_1(\mathbf{PS}_1)))$ by the definition of sequential composition (Theorem 3.4). $\square$

Proposition 4.2 (Symmetry of parallel composition). *For moves m_1 targeting c_1 and m_2 targeting c_2 with $c_1 \cap c_2 = \emptyset$, we have $m_1 \otimes m_2 = m_2 \otimes m_1$.*

Proof. Union is commutative, and disjointness ensures no interference. $\square$

Proposition 4.3 (Interchange law). *If m_1, m_2 target disjoint coordinates and m_3, m_4 target the same disjoint coordinates respectively, then $(m_3 \circ m_1) \otimes (m_4 \circ m_2) = (m_3 \otimes m_4) \circ (m_1 \otimes m_2)$.*

Proof. Both sides apply m_1 to c_1 , m_2 to c_2 , then m_3 to c_1 , m_4 to c_2 . Disjointness ensures independence. $\square$

4.2 Idempotency properties

Lemma 4.4 (Idempotency of `add_obstruction`). *If $b \in \mathcal{B}(c)$ already, then `add_obstruction`(c, b) is the identity: `add_obstruction`(c, b)(PS) = PS .*

Proof. Adding an element to a set that already contains it yields the same set. $\square$

Lemma 4.5 (Idempotency of `strengthen`). *If $\varphi' = \varphi$ then `strengthen`(φ, φ') = id_{PS} .*

Lemma 4.6 (Involution of double restriction). *For $c'' \subseteq c' \subseteq c$, we have `restrict_to`(c'') $\circ$ `restrict_to`(c') = `restrict_to`(c'').*

Proof. By the transitivity of sheaf restriction: $\text{PS}|_{c'}|_{c''} = \text{PS}|_{c''}$. $\square$

Lemma 4.7 (Resolve-add cancellation). *If `resolve_obstruction`(c, b) succeeds, then `add_obstruction`(c, b) $\circ$ `resolve_obstruction`(c, b)(PS) re-introduces b , but `resolve_obstruction`(c, b) $\circ$ `add_obstruction`(c, b)(PS) = PS provided the resolution evidence is still valid.*

4.3 Classical–geometric commutation

The interaction between classical and geometric moves is governed by the following commutation lemmas.

Lemma 4.8 (Classical moves commute with disjoint restrictions). *If m is a classical move targeting coordinate c and $c' \cap c = \emptyset$, then $m \circ \text{restrict_to}(c') = \text{restrict_to}(c') \circ m$.*

Proof. Classical moves only modify evidence and obligations at their target coordinate. Restriction to a disjoint coordinate c' does not affect evidence at c , so the operations commute. $\square$

Lemma 4.9 (Strengthen commutes with merge). *If `strengthen`(φ, φ') acts on coordinate c_1 and `merge`(c_1, c_2) glues along $c_1 \cap c_2$, then `strengthen-then-merge` yields the same result as `merge-then-strengthen` on the merged coordinate, provided the strengthening is compatible with the overlap.*

Lemma 4.10 (Transport distributes over composition). *For morphisms $f : c_1 \rightarrow c_2$ and $g : c_2 \rightarrow c_3$ in $\mathcal{S}$, `transport_along`($g \circ f$) = `transport_along`(g) $\circ$ `transport_along`(f).*

Proof. By the functoriality of the pushforward: $(g \circ f)_* = g_* \circ f_*$. $\square$

Lemma 4.11 (Descent absorbs merge). *If a descent move succeeds on a cover $\{U_i\}$, then any prior merge moves on overlaps $U_i \cap U_j$ are subsumed: descent produces a global section that is consistent with all local data, making the merge results redundant.*

5 Move Taxonomy

Definition 5.1 (Move kinds). The 13 move kinds are partitioned into 6 classical and 7 geometric: **Classical** (have LEAN 4/COQ analogues): `compose`, `split`, `strengthen`, `weaken`, `add_obligation`, `discharge_obligation`.

Geometric (no LEAN 4/COQ analogue): `restrict_to`, `merge`, `transport_along`, `add_obstruction`, `resolve_obstruction`, `repair`, `descend`.

Theorem 5.2 (Geometric inexpressibility). *The 7 geometric moves cannot be simulated by any finite combination of LEAN 4 4, Ltac2, or Meta-F* tactics.*

Table 1: The 10 named moves and their formal kinds.

Move	Formal Kind(s)	Category	Trust Effect
VERIFY	discharge_obligation	Classical	Preserves/raises
CONSTRUCT	compose, add_obligation	Classical	Sets initial
NORMALIZE	strengthen, weaken	Classical	Preserves
GLUE	merge	Geometric	Conservative join
RESTRICT	restrict_to	Geometric	Monotone ($\preceq$)
TRANSPORT	transport_along	Geometric	Monotone ($\preceq$)
REPAIR	repair	Geometric	May raise
PROMOTE	(trust algebra)	Trust	Raises w/ justification
CHALLENGE	add/resolve_obstruction	Geometric	May lower
ESCALATE	descend	Geometric	Backend ceiling

Proof. We prove for three representative moves:

descend: requires computing a limit over a covering diagram. LEAN 4’s state has no covering families.

repair: modifies the Grothendieck topology. LEAN 4’s state has no topology component.

transport_along: requires a pushforward functor f_* along a site morphism. LEAN 4’s **rw** rewrites along equalities, not along categorical morphisms.

The argument extends to Ltac2 and Meta-F*.

□

6 The 10 Named Moves

The 13 formal move kinds map to 10 named moves in the JUGEOR orchestration layer:

Table 1 shows each move with its formal kind(s), category, and trust effect. The three classical moves (VERIFY, CONSTRUCT, NORMALIZE) correspond to standard tactic operations. The five geometric moves (GLUE, RESTRICT, TRANSPORT, REPAIR, ESCALATE) operate on site structure. PROMOTE and CHALLENGE manipulate trust directly via the trust algebra of Paper 4.

We now describe each move in detail.

6.1 Verify

Definition 6.1 (VERIFY). $\text{VERIFY}(c, \varphi, \text{src})$ discharges obligation φ at coordinate c using evidence from source src .

Precondition. $\varphi \in \mathcal{O}(c)$ and a source channel $\text{src} \in \text{SrcChan}$ is available.

Postcondition. $\varphi \notin \mathcal{O}'(c)$ and $\exists e \in \mathcal{E}'(c)$ with $e \vdash \varphi$.

Trust effect. $\mathcal{T}(\varphi)$ is set to the trust level of src : SOLVER for an SMT solver, RUNTIME for runtime testing, PROOF for a formal proof term.

Example. For a function $\text{factorial}(n)$, the obligation $\varphi = \text{factorial}(n) \geq 1$ at coordinate $c_{\text{factorial}}$ is verified by the SMT solver, yielding evidence at trust level SOLVER.

6.2 Construct

Definition 6.2 (CONSTRUCT). $\text{CONSTRUCT}(c, \text{spec})$ builds a new judgment at coordinate c from specification spec , generating obligations.

Precondition. $c \in \text{Ob}(\mathcal{S})$ and spec is a well-formed specification.

Postcondition. A new judgment j is added to $\mathcal{J}(c)$ with obligations $\{\varphi_1, \dots, \varphi_k\} \subseteq \mathcal{O}'(c)$.

Trust effect. Initial trust is UNVERIFIED for all generated obligations.

Example. Constructing the specification of an LRU cache class generates obligations for capacity invariants, eviction ordering, and thread-safety annotations at the class coordinate.

6.3 Normalize

Definition 6.3 (NORMALIZE). $\text{NORMALIZE}(c, \varphi)$ canonicalizes the representation of obligation φ at coordinate c via strengthening or weakening.

Precondition. $\varphi \in \mathcal{O}(c)$.

Postcondition. φ is replaced by a logically equivalent φ' in canonical form.

Trust effect. Preserved—normalization does not change evidence quality.

Example. The obligation $x > 0 \wedge x > 0$ is normalized to $x > 0$ by idempotency of conjunction.

6.4 Glue

Definition 6.4 (GLUE). $\text{GLUE}(c_1, c_2)$ merges local proof states $\text{PS}|_{c_1}$ and $\text{PS}|_{c_2}$ along the overlap $c_1 \cap c_2$.

Precondition. $c_1, c_2 \in \text{Ob}(\mathcal{S})$ with $c_1 \cap c_2 \neq \emptyset$, and $\mathcal{E}|_{c_1|_{c_1 \cap c_2}} = \mathcal{E}|_{c_2|_{c_1 \cap c_2}}$ (overlap compatibility).

Postcondition. A new coordinate $c_{12} = c_1 \cup c_2$ with merged evidence $\mathcal{E}'(c_{12})$ supporting $\mathcal{J}(c_{12})$.

Trust effect. Conservative join: $\mathcal{T}' = \mathcal{T}(\mathcal{E}|_{c_1}) \oplus \mathcal{T}(\mathcal{E}|_{c_2})$.

Example. In a multi-function program where `parse()` and `validate()` share an interface type `Token`, GLUE merges their proof states along the `Token` overlap, producing a combined proof that respects the shared interface contract.

6.5 Restrict

Definition 6.5 (RESTRICT). $\text{RESTRICT}(c, c')$ restricts the proof state from coordinate c to a sub-coordinate $c' \subseteq c$.

Precondition. $c' \subseteq c$ in the partial order of $\text{Ob}(\mathcal{S})$.

Postcondition. $\text{PS}' = \text{PS}|_{c'}$ with all components restricted.

Trust effect. Monotone: $\mathcal{T}(\mathcal{E}|_{c'}) \preceq \mathcal{T}(\mathcal{E}|_c)$.

Example. From a module-level proof state, RESTRICT narrows focus to a single class, allowing targeted verification of class invariants.

6.6 Transport

Definition 6.6 (TRANSPORT). $\text{TRANSPORT}(f)$ transports proof data along a morphism $f : c_1 \rightarrow c_2$ in $\mathcal{S}$ using the pushforward functor f_* .

Precondition. $f \in \text{Mor}(\mathcal{S})$ is an admissible morphism with a defined pushforward.

Postcondition. Evidence at c_1 is transported to c_2 : $\mathcal{E}'(c_2) = f_*(\mathcal{E}(c_1))$.

Trust effect. Monotone: $\mathcal{T}(f_*(e)) \preceq \mathcal{T}(e)$ by functoriality.

Example. When a helper function `gcd(a, b)` is called inside `simplify_fraction(n, d)`, transport along the call morphism carries the GCD correctness proof from the callee to the caller coordinate.

6.7 Repair

Definition 6.7 (REPAIR). $\text{REPAIR}(c)$ refines the covering family $\text{Cov}(c)$ to eliminate an obstruction in $\mathcal{B}(c)$.

Precondition. $\mathcal{B}(c) \neq \emptyset$ and the covering family is refinable.

Postcondition. $\text{Cov}'(c)$ is a refinement of $\text{Cov}(c)$ and $|\mathcal{B}'(c)| < |\mathcal{B}(c)|$.

Trust effect. May raise trust if the refinement enables new evidence.

Example. If gluing fails for a protocol handler because two sub-modules disagree on a timeout parameter, REPAIR refines the cover by splitting the timeout-sensitive region into a separate coordinate, allowing independent verification.

6.8 Promote

Definition 6.8 (PROMOTE). $\text{PROMOTE}(c, \varphi, \tau')$ raises the trust level of obligation φ at coordinate c to τ' with justification.

Precondition. $\mathcal{T}(\varphi) \preceq \tau'$ and a justification is provided (e.g., a formal proof term, a successful test suite).

Postcondition. $\mathcal{T}'(\varphi) = \tau'$.

Trust effect. Raises trust with justification.

Example. A runtime-tested invariant (RUNTIME) is promoted to PROOF after a LEAN 4 proof term is supplied.

6.9 Challenge

Definition 6.9 (CHALLENGE). $\text{CHALLENGE}(c, \varphi)$ introduces or resolves an obstruction for obligation φ at coordinate c .

Precondition. φ is assigned to c with non-trivial evidence.

Postcondition. Either $\mathcal{B}'(c) = \mathcal{B}(c) \cup \{b_\varphi\}$ (obstruction added) or $\mathcal{B}'(c) = \mathcal{B}(c) \setminus \{b_\varphi\}$ (obstruction resolved).

Trust effect. May lower trust if an obstruction is added; may raise if resolved.

Example. A counterexample found by property-based testing challenges the assumed postcondition of a sort function, adding an obstruction that must be resolved before the proof can proceed.

6.10 Escalate

Definition 6.10 (ESCALATE). $\text{ESCALATE}(\{U_i\}, \{s_i\})$ performs sheaf descent over a covering family $\{U_i\}$ with compatible local sections $\{s_i\}$.

Precondition. $\{U_i\}$ covers the target coordinate, each $s_i \in \mathcal{E}(U_i)$, and the cocycle condition holds on overlaps.

Postcondition. A global section s with $s|_{U_i} = s_i$ for all i , or a descent obstruction in H^1 .

Trust effect. Trust is bounded by the backend ceiling: $\mathcal{T}(s) \leq \min_i \mathcal{T}(s_i)$.

7 Rule and Transition Coverage

Definition 7.1 (Rule kinds). The rule-kind enum: STRUCTURAL, SEMANTIC, AXIOM, DERIVED.

Definition 7.2 (Transition kinds). The transition-kind enum: FORWARD, BACKWARD, LATERAL, FIXPOINT, INTERRUPT.

Lemma 7.3 (Coverage completeness). *The 13 move kinds span the full $\text{RuleKind} \times \text{TransKind}$ product space: all $4 \times 5 = 20$ cells are covered.*

8 The Adaptive Control Layer

Definition 8.1 (Control strategies). The control-strategy enum: GREEDY, BEAM_SEARCH, MCTS, GRADIENT_DESCENT.

Definition 8.2 (Obstruction norm). $\|\mathcal{B}\| = |\{c \in \text{Ob}(\mathcal{S}) : \mathcal{B}(c) \neq 0\}|$.

Theorem 8.3 (Controller termination). *The control loop terminates when $\|\mathcal{B}\|$ is minimized under any strategy with a finite move budget.*

9 The Construction Loop

The construction loop has four phases:

1. **PROPOSE**: generate candidate local sections via source channels (solver, oracle, runtime).
2. **NORMALIZE**: canonicalize proposed sections and check local well-formedness.
3. **COMPARE**: evaluate candidates against the obstruction norm and trust floor.
4. **SELECT**: choose the best candidate and apply the corresponding semantic move.

10 Deep JuGeo API Reference

10.1 The MoveKind enum

Listing 1: MoveKind and SemanticMove.

```

1 from jugeo.orchestration.controller import (
2     MoveKind, SemanticMove)
3
4 # MoveKind enum values:
5 # VERIFY, CONSTRUCT, REPAIR, NEGOTIATE_TREATY,
6 # REFINE_COVER, DISCHARGE_OBLIGATION, CONSULT_ORACLE
7 # Extended: NORMALIZE, GLUE, RESTRICT, TRANSPORT,
8 #           PROMOTE, CHALLENGE, ESCALATE
9
10 # SemanticMove attributes:
11 # move_id, kind, target_coordinate,
12 # expected_gain, estimated_cost,
13 # preconditions, postconditions

```

10.2 Coordinates and morphisms

Listing 2: Coordinate and Morphism API.

```

1 from jugeo.geometry.site import (
2     Coordinate, CoordinateKind,

```

```

3     Morphism, MorphismKind)
4
5 # CoordinateKind: MODULE, FUNCTION, INTERFACE,
6 #                 TEST, THEOREM, REGION
7 # Coordinate methods: parent(), children(),
8 #   is_prefix_of(), common_ancestor(), distance_to()
9
10 # MorphismKind: RESTRICTION, INCLUSION,
11 #              TRANSPORT, REFINEMENT
12 # Morphism methods: compose(), is_identity,
13 #   is_invertible(), reversed(), factors_through()

```

10.3 Descent engine

Listing 3: DescentEngine API.

```

1 from jugeo.geometry.descent import (
2     DescentEngine, DescentResult,
3     DescentStrategy)
4
5 # DescentStrategy: EAGER, EXHAUSTIVE,
6 #                 ITERATIVE, OPTIMISTIC
7 engine = DescentEngine(strategy=DescentStrategy.EAGER)
8 result = engine.run(cover, sections)
9 # result is GlobalSection or DescentObstruction

```

10.4 Complete SemanticMove construction

The following example shows end-to-end construction and application of a semantic move:

Listing 4: Constructing and applying a SemanticMove.

```

1 from jugeo.orchestration.controller import (
2     MoveKind, SemanticMove, ProofState)
3 from jugeo.geometry.site import Coordinate, CoordinateKind
4
5 # 1. Build the target coordinate
6 coord = Coordinate(
7     name="validate_input",
8     kind=CoordinateKind.FUNCTION,
9     path="validators.py:validate_input"
10 )
11
12 # 2. Define pre- and post-conditions
13 pre = lambda ps: coord in ps.site.objects
14 post = lambda ps: ps.obligations[coord] == set()
15
16 # 3. Construct the semantic move
17 move = SemanticMove(
18     move_id="sm-verify-validate-001",
19     kind=MoveKind.VERIFY,
20     target_coordinate=coord,
21     expected_gain=0.85,

```

```

22     estimated_cost=0.12,
23     preconditions=[pre],
24     postconditions=[post],
25     trust_floor="SOLVER"
26 )
27
28 # 4. Apply the move to a proof state
29 ps_new = move.apply(proof_state)
30 assert move.postconditions_met(ps_new)
31 print(f"Trust: {ps_new.trust[coord]}")
32 # Output: Trust: SOLVER

```

10.5 DescentEngine run with output

Listing 5: DescentEngine end-to-end run.

```

1 from jugeo.geometry.descent import (
2     DescentEngine, DescentStrategy)
3 from jugeo.geometry.site import (
4     SemanticSite, CoveringFamily)
5
6 # Build a site for a multi-function program
7 site = SemanticSite.from_program("etl_pipeline.py")
8 cover = CoveringFamily(
9     base=site.root,
10    patches=[
11        site.coordinate("extract"),
12        site.coordinate("transform"),
13        site.coordinate("load"),
14    ]
15 )
16
17 # Collect local sections (evidence per function)
18 sections = {
19     c: site.evidence(c)
20     for c in cover.patches
21 }
22
23 # Run descent
24 engine = DescentEngine(
25     strategy=DescentStrategy.EAGER)
26 result = engine.run(cover, sections)
27
28 if result.succeeded:
29     print(f"Global section: {result.section}")
30     print(f" $H^1$  = {result.cohomology}")
31     # Output: Global section: <GlobalSection ...>
32     # Output:  $H^1$  = 0
33 else:
34     print(f"Obstruction: {result.obstruction}")
35     print(f"Repair hint: {result.repair_hint}")

```

10.6 Coordinate and Morphism creation

Listing 6: Coordinate and Morphism creation and composition.

```
1 from jugео.geometry.site import (
2     Coordinate, CoordinateKind,
3     Morphism, MorphismKind)
4
5 # Create coordinates for a class hierarchy
6 mod = Coordinate("cache_module", CoordinateKind.MODULE,
7                 "cache.py")
8 cls = Coordinate("LRUCache", CoordinateKind.FUNCTION,
9                 "cache.py:LRUCache")
10 mth = Coordinate("get", CoordinateKind.FUNCTION,
11                 "cache.py:LRUCache.get")
12
13 # Create morphisms (inclusion, restriction)
14 incl_cls = Morphism(
15     source=cls, target=mod,
16     kind=MorphismKind.INCLUSION,
17     label="class-to-module")
18 incl_mth = Morphism(
19     source=mth, target=cls,
20     kind=MorphismKind.INCLUSION,
21     label="method-to-class")
22
23 # Compose morphisms
24 incl_composed = incl_cls.compose(incl_mth)
25 assert incl_composed.source == mth
26 assert incl_composed.target == mod
27
28 # Create a restriction (reverse direction)
29 restr = incl_cls.reversed()
30 assert restr.kind == MorphismKind.RESTRICTION
31 assert restr.source == mod
32 assert restr.target == cls
33
34 # Check factoring
35 assert incl_composed.factors_through(cls)
36 print(f"Distance: {mth.distance_to(mod)}")
37 # Output: Distance: 2
```

11 Evaluation

We evaluate semantic moves on 100 Python programs across four categories, each containing 25 programs. All numbers are produced by the `exp06_semantic_moves.py` experiment with random seed 42.

11.1 Experimental setup

The four categories are:

Table 2: Morphism analysis by program category (25 programs each).

Category	Avg C	Avg M	M/C	M>C	% M>C
pure_function	2.1	1.3	0.62	2/25	8%
multi_function	4.2	5.8	1.38	19/25	76%
class	5.6	7.2	1.29	18/25	72%
module_w_deps	6.8	9.4	1.38	22/25	88%
Total (100)	4.7	5.9	1.26	61/100	61%

- **pure_function** (25): single or multi-step mathematical functions (Fibonacci, factorial, matrix operations, prime sieve).
- **multi_function** (25): multiple functions with interdependencies (ETL pipelines, search-rank systems, validation chains).
- **class** (25): object-oriented classes (Stack, Queue, LRUCache, Trie, BloomFilter, StateMachine).
- **module_with_deps** (25): modules with internal dependencies (test frameworks, schema validators, config managers, protocol handlers).

Each program is processed by `jugeo prove` and `jugeo encode`. The soundness proof ([proofs/jugeo/paper06](#)) verifies that morphisms exist for all program types, that multi-function sites have richer morphism structure, and that descent is effective ($H^1 = 0$).

11.2 Morphism counts by category

Table 2 shows that for `multi_function`, `class`, and `module_with_deps` programs, morphisms outnumber coordinates ($M/C > 1$), confirming that richer program structure demands the geometric apparatus. For pure functions, $M/C = 0.62$ because simple functions have few inter-coordinate relationships.

11.3 Semantic move composition

The geometric fraction is $458/1236 = 37.1\%$ (Table 3). Among geometric moves, `add_obstruction` and `resolve_obstruction` account for 142 applied each (bookkeeping), while `merge` (68), `transport_along` (38), `descend` (28), `restrict_to` (24), and `repair` (16) reflect genuine geometric proof activity.

11.4 Per-category move analysis

Table 4 reveals a clear trend: pure functions need mostly classical moves (19.6% geometric), while multi-function and structured programs require $> 36\%$ geometric moves. The gap is driven by `merge`, `transport_along`, and `descend`, which fire when composing judgments across function calls, class methods, or module imports.

11.5 Morphisms $>$ coordinates

Table 5 confirms that morphisms exceed coordinates for the majority of non-trivial programs. For pure functions, only 8% have $M > C$; for modules, 88% have $M > C$. This validates the judgment geometry thesis: for structured programs, the morphism structure (function calls, imports, class inheritance) is richer than the coordinate structure (code regions).

Table 3: Aggregate move composition over 100 programs.

Move Kind	Cat.	Applied	Failed	Lean 4 Equiv.
compose	C	68	0	apply/exact
split	C	142	0	split
strengthen	C	142	0	assumption
weaken	C	142	0	sorry
add_obligation	C	142	0	have
discharge_obligation	C	142	0	exact
restrict_to	G	24	8	—
merge	G	68	12	—
transport_along	G	38	234	—
add_obstruction	G	142	0	—
resolve_obstruction	G	142	0	—
repair	G	16	86	—
descend	G	28	64	—
Classical		778	0	
Geometric		458	404	
Total		1236	404	

Table 4: Move statistics by category.

Category	Total	Classical	Geometric	Geo%
pure_function	189	152	37	19.6%
multi_function	312	198	114	36.5%
class	367	214	153	41.7%
module_w_deps	368	214	154	41.8%
Total	1236	778	458	37.1%

11.6 Coverage matrix

All 20 cells are covered (Table 6).

11.7 Comparison with Lean 4 tactics

11.8 Timing

Total elapsed time for 100 programs: 331.88s. Mean prove time per program: 4.8s (eager strategy). Semantic site construction ranges from 70–520 μ s. Move-test times: 190–2450 μ s. These microsecond-scale times confirm negligible overhead.

11.9 Failed move analysis

Of 404 failed move attempts across 100 programs:

- **transport_along**: 234 (57.9%)—requires admissible morphism with defined pushforward.
- **repair**: 86 (21.3%)—requires refinable covering family.
- **descend**: 64 (15.8%)—requires compatible family over a cover.
- **merge**: 12 (3.0%)—overlap compatibility check.

Table 5: Programs where morphisms exceed coordinates.

Category	M>C	M=C	M<C	% M>C
pure_function	2	3	20	8%
multi_function	19	3	3	76%
class	18	4	3	72%
module_w_deps	22	2	1	88%

Table 6: RuleKind $\times$ TransKind coverage matrix. $\checkmark$ = at least one move sequence covers the cell.

	FWD	BWD	LAT	FIX	INT
STRUCT	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
SEMAN	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
AXIOM	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
DERIV	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

- `restrict_to`: 8 (2.0%)—coordinate not in site.

11.10 Descent analysis

We analyze descent success rates broken down by program category and descent strategy.

Table 8 reveals several patterns. First, pure functions always succeed because they have trivial covering families (a single coordinate covers the entire site). Second, the exhaustive strategy achieves the highest success rate (84%) by exploring all possible compatible families before reporting an obstruction, at the cost of higher computation time. Third, the optimistic strategy has the lowest success rate (71%) because it assumes compatibility without checking, and must backtrack when the cocycle condition fails.

The gap between eager (74%) and exhaustive (84%) strategies grows with program complexity: for modules, the gap is 16 percentage points, suggesting that complex programs benefit from more thorough descent exploration. The iterative strategy (81%) provides a good compromise, using bounded iteration to refine incompatible families before giving up.

When descent fails, the obstruction is always in $H^1(\mathcal{S}, \mathcal{J})$, and the most common cause is incompatible evidence on overlaps—typically arising from functions that share mutable state or modules with circular dependencies. The REPAIR move can address 62% of these failures by refining the covering family to separate the conflicting regions.

11.11 Threats to validity

Internal validity. Our evaluation uses synthetic benchmarks generated from representative Python patterns. While the 100 programs span four categories and include both simple and complex structures, they may not capture all patterns found in production codebases. The random seed (42) is fixed for reproducibility, but different seeds might shift aggregate statistics by 2–5 percentage points.

External validity. All programs are in Python; the geometric fraction and morphism-to-coordinate ratios may differ for languages with different module systems (e.g., Haskell’s type classes, Rust’s

Table 7: Comparison with LEAN 4 4 tactics.

Metric	Lean 4 4	Sem. Moves	Diff
Simple lemma	3	7–12	+4–9
Medium lemma	8	12–27	variable
Complex lemma	18	27–36	+9–18
Geometric fraction	0%	37.1%	+37.1pp
Move kinds	~6	13	+7
Trust manipulation	No	Yes	—
Obstruction tracking	No	Yes	—
Sheaf descent	No	Yes	—

Table 8: Descent success rates by category and strategy (percentage of descent attempts that produce a global section).

Category	Eager	Exhaustive	Iterative	Optimistic
pure_function	100%	100%	100%	100%
multi_function	72%	84%	80%	68%
class	68%	80%	76%	64%
module_w_deps	56%	72%	68%	52%
Average	74%	84%	81%	71%

trait system). We have not evaluated on programs larger than ~500 lines, where the geometric fraction is expected to increase further.

Construct validity. The “geometric fraction” metric counts geometric moves as a proportion of total moves. This metric treats all moves equally regardless of their computational cost or proof significance. An alternative weighting by trust gain or evidence quality might yield different conclusions about the importance of geometric apparatus.

Conclusion validity. Our sample of 25 programs per category provides moderate statistical power. For the key claim that the geometric fraction increases with program complexity, a Kruskal–Wallis test yields $p < 0.001$, confirming statistical significance despite the moderate sample size.

12 Comparison with Existing Tactic Systems

12.1 Feature matrix

12.2 Tactic simulation analysis

Of the 13 move kinds, the 6 classical moves can be simulated: `compose` $\leftrightarrow$ `apply/exact`, `split` $\leftrightarrow$ `split/cases`, `strengthen` $\leftrightarrow$ `assumption`, `weaken` $\leftrightarrow$ `sorry/admit`, `add_obligation` $\leftrightarrow$ `have/suffices`, `discharge_obligation` $\leftrightarrow$ `exact`.

The 7 geometric moves cannot be simulated (Theorem 5.2). We now give a detailed proof sketch for each geometric move.

Table 9: Feature comparison of tactic frameworks.

Feature	Lean 4 4	Ltac2	Meta-F*	Sem. Moves
Typed tactics	✓	✓	✓	✓
Sequential comp.	✓	✓	✓	✓
Parallel comp.				✓
Backtracking	✓	✓		✓
SMT integration			✓	✓
Trust manipulation				✓
Coordinate-aware				✓
Cover-aware				✓
Obstruction tracking				✓
Sheaf descent				✓
Repair generation				✓

12.2.1 Inexpressibility of `restrict_to`

The `restrict_to` move narrows the proof state to a sub-coordinate, restricting all components (evidence, obligations, obstructions, trust) simultaneously. In LEAN 4, one can focus on a subgoal via `case` or `show`, but these operate on the proof *term*, not on a coordinate in a semantic site. There is no LEAN 4 tactic that restricts trust annotations, obstruction presheaves, or covering families to a sub-site. The state components that `restrict_to` modifies (site topology, trust map, obstruction presheaf) have no representation in the LEAN 4 metavariable context.

12.2.2 Inexpressibility of `merge`

The `merge` move computes a pushout of two local proof states along their overlap. This requires: (i) an overlap intersection $c_1 \cap c_2$ in the site; (ii) a compatibility check on evidence over the overlap; (iii) a conservative join of trust levels. LEAN 4’s `simp` and `ring` can combine lemmas, but they operate within a single proof context, not across two independent proof states with their own trust levels and obstruction data. The merge operation is a categorical colimit, and LEAN 4’s tactic monad does not support colimit computation over proof contexts.

12.2.3 Inexpressibility of `transport_along`

Transport applies a pushforward functor f_* along a site morphism $f : c_1 \rightarrow c_2$. While LEAN 4’s `rw` tactic rewrites along an equality $a = b$, it does not transport along a categorical morphism between different coordinates. The pushforward requires tracking how evidence, obligations, and trust transform covariantly along f —a functorial operation that has no analogue in term rewriting. Moreover, `rw` does not modify trust levels, which f_* does by the monotonicity property.

12.2.4 Inexpressibility of `add_obstruction` and `resolve_obstruction`

These moves modify the obstruction presheaf $\mathcal{B}$, a component that does not exist in any existing tactic framework. Adding an obstruction records a cohomological obstruction at a coordinate; resolving one removes it by providing evidence. LEAN 4’s closest analogue is `sorry`, which admits a goal without evidence, but `sorry` does not record an obstruction—it silently assumes the goal is

true. The obstruction presheaf enables the system to track *why* a proof attempt failed and *where* the failure occurred in the site topology, information that `sorry` discards.

12.2.5 Inexpressibility of repair

Repair modifies the Grothendieck topology itself—refining covering families to eliminate obstructions. No tactic in `LEAN 4`, `Ltac2`, or `Meta-F*` can modify the topology of the proof state because the proof state has no topology. The closest analogue in `LEAN 4` is changing the set of hypotheses via `have` or `obtain`, but these add hypotheses to a single proof context rather than refining a covering family over a topological space.

12.2.6 Inexpressibility of descend

Descent computes a global section from compatible local data by the sheaf axiom. This requires: (i) a covering family; (ii) local sections satisfying the cocycle condition on overlaps; (iii) the unique gluing provided by the sheaf property. `LEAN 4`’s proof state has no covering families, no overlap computations, and no sheaf axiom. While one could manually encode the descent computation as a `LEAN 4` proof, this would require the user to implement sheaf theory inside `LEAN 4`—the descent operation would not be a *tactic* but rather a *theorem*, and the proof search automation provided by the `descend` move would be lost.

12.3 Expressiveness ordering

$$\text{LEAN 4} \equiv_{\text{exp}} \text{Ltac2} \equiv_{\text{exp}} \text{Meta-F}^* \subsetneq_{\text{exp}} \text{Semantic Moves}$$

The gap is the 7 geometric move kinds.

13 Related Work and Conclusion

13.1 Related work

Tactic frameworks. `LEAN 4` [1], `Ltac2` [3], and `Meta-F*` [4] are closest. `Mtac2` [9] and `Eisbach` [10] add typed meta-programming and method languages but remain term-based.

Proof search and automation. Hammer tools [11] erase geometric structure. ML-guided search [12, 13] could benefit from 7 additional geometric move kinds as an action space.

Sheaf-theoretic methods. Abramsky et al. [16] use sheaves for contextuality in quantum mechanics. Our application to proof search is the first use of Grothendieck topologies for tactic design.

Compositional verification. Hoare logic, rely-guarantee, and separation logic compose proofs within a fixed calculus. Semantic moves add topological composition via covering families.

Proof repair. Ringer et al. [17] repair proof terms. Our `repair` move modifies the site structure, a complementary approach.

Program synthesis. Program synthesis systems such as Sketch [18] and Rosette [19] construct programs from specifications using counterexample-guided inductive synthesis (CEGIS). While CEGIS iteratively refines candidates, it operates on a flat search space of program terms without geometric structure. Semantic moves could augment synthesis by providing a richer action space: the CONSTRUCT move generates candidate local sections, while GLUE and ESCALATE compose them across module boundaries—operations that CEGIS cannot express because it lacks a notion of covering families or overlap compatibility.

Proof planning. Bundy’s proof planning framework [20] organizes proof search around high-level *methods* that encode common proof patterns. Methods have preconditions, postconditions, and effects—similar to our semantic moves. However, proof planning methods operate on proof terms and do not manipulate trust levels, site structure, or obstructions. Our move taxonomy can be viewed as an extension of proof planning to a geometric setting, where methods additionally modify the topology of the proof state and where the control layer uses obstruction norms rather than heuristic method selection.

Rippling. Rippling [21] is a heuristic for guiding rewriting in inductive proofs by annotating terms with *wave fronts* and directing rewrites toward the induction hypothesis. While rippling provides directed search within a single proof context, semantic moves provide directed search across *multiple* proof contexts linked by site morphisms. The TRANSPORT move is the geometric analogue of rippling: both direct proof progress along a structural relationship, but transport operates along categorical morphisms rather than syntactic wave fronts.

Strategy languages. Strategy languages such as Stratego [22] and Elan [23] provide programmable term-rewriting strategies with combinators for sequencing, choice, and recursion. These languages are Turing-complete over term transformations but cannot express geometric operations: there is no combinator for restricting to a sub-site, gluing along an overlap, or performing sheaf descent. Semantic moves extend the strategy language paradigm from term rewriting to site-level proof manipulation.

Proof by analogy. Proof by analogy [24] transfers proof structure from a solved problem to a new one via a mapping between their representations. Our TRANSPORT move formalizes a restricted form of proof analogy: transporting evidence along a site morphism $f : c_1 \rightarrow c_2$ is analogous to mapping proof structure from c_1 to c_2 . However, TRANSPORT is functorial and trust-aware, properties that the informal analogy framework of Melis lacks. Moreover, semantic moves compose analogies via the move algebra (Section 4), enabling iterated transport along chains of morphisms.

13.2 Conclusion

We introduced semantic moves—typed, composable proof-search operations acting on the full geometric proof state. Of 13 move kinds, 7 are geometric and have no LEAN 4/COQ/F* analogue.

Evaluation on 100 programs across four categories shows:

- Geometric fraction rises from 19.6% (pure functions) to 41.8% (modules).
- Morphisms exceed coordinates for 76% of multi-function programs and 88% of modules.
- All 20 rule–transition cells are covered.
- The 10 named moves (VERIFY, CONSTRUCT, NORMALIZE, GLUE, RESTRICT, TRANSPORT, REPAIR, PROMOTE, CHALLENGE, ESCALATE) provide a complete orchestration vocabulary.

Future work: larger programs where `transport_along` and `repair` succeed more frequently, ML-guided move selection, and a LEAN 4 bridge for classical-move export.

A Representative Programs

Table 10 lists 20 representative programs from our evaluation, showing their category, coordinate and morphism counts, geometric fraction, and key moves used.

Table 10: 20 representative programs with structural and move statistics.

Program	Cat.	C	M	Geo%	Key Moves
<code>fibonacci</code>	PF	1	0	0.0%	VERIFY, CONSTRUCT
<code>factorial</code>	PF	1	0	14.3%	VERIFY, NORMALIZE
<code>matrix_multiply</code>	PF	2	1	16.7%	VERIFY, RESTRICT
<code>prime_sieve</code>	PF	2	1	20.0%	CONSTRUCT, VERIFY
<code>binary_search</code>	PF	1	0	0.0%	VERIFY
<code>etl_pipeline</code>	MF	4	6	38.5%	GLUE, TRANSPORT
<code>search_rank</code>	MF	5	8	42.1%	TRANSPORT, ESCALATE
<code>validation_chain</code>	MF	3	4	33.3%	GLUE, VERIFY
<code>parser_tokenizer</code>	MF	4	5	35.7%	RESTRICT, GLUE
<code>event_dispatcher</code>	MF	5	7	40.0%	TRANSPORT, GLUE
<code>stack</code>	CL	4	5	33.3%	CONSTRUCT, GLUE
<code>lru_cache</code>	CL	6	9	44.4%	GLUE, REPAIR
<code>trie</code>	CL	5	7	41.7%	TRANSPORT, GLUE
<code>bloom_filter</code>	CL	4	5	36.4%	VERIFY, CHALLENGE
<code>state_machine</code>	CL	7	10	47.1%	ESCALATE, REPAIR
<code>test_framework</code>	MD	8	12	44.0%	ESCALATE, GLUE
<code>schema_validator</code>	MD	6	8	40.0%	REPAIR, TRANSPORT
<code>config_manager</code>	MD	5	7	38.5%	GLUE, RESTRICT
<code>protocol_handler</code>	MD	8	13	46.2%	ESCALATE, REPAIR
<code>plugin_loader</code>	MD	7	10	42.9%	TRANSPORT, ESCALATE

Cat.: PF = pure_function, MF = multi_function, CL = class, MD = module_with_deps. C = coordinate count, M = morphism count, Geo% = geometric fraction of moves applied.

Several patterns are visible. Pure functions (`fibonacci`, `binary_search`) have 0–2 coordinates, 0–1 morphisms, and low or zero geometric fractions. Multi-function programs (`etl_pipeline`, `search_rank`) consistently show $M > C$ and geometric fractions above 33%. Classes (`lru_cache`, `state_machine`) exhibit the highest geometric fractions among non-module programs because class invariants create rich overlap structure between methods. Modules (`protocol_handler`, `test_framework`) have the most morphisms and the highest geometric fractions, driven by cross-module transport and descent operations.

References

- [1] L. de Moura et al. The Lean 4 theorem prover. In *CADE-28*, Springer LNCS, 2021.
- [2] J. Avigad et al. The hitchhiker’s guide to logical verification, 2023.
- [3] P.-M. Pédrot. Ltac2: Tactical warfare. In *CoqPL Workshop*, 2019.

- [4] G. Martínez et al. Meta-F*: Proof automation with SMT, tactics, and metaprograms. In *ESOP*, Springer LNCS, 2019.
- [5] H. Young. Semantic sites for program verification. Judgment Geometry Paper 1, 2024.
- [6] H. Young. The judgment algebra. Judgment Geometry Paper 2, 2024.
- [7] H. Young. Descent and obstructions. Judgment Geometry Paper 3, 2024.
- [8] H. Young. Trust algebras for multi-source verification. Judgment Geometry Paper 4, 2024.
- [9] J.-O. Kaiser et al. Mtac2: Typed tactics for backward reasoning in Coq. In *ICFP*, ACM, 2018.
- [10] D. Matichuk, T. Murray, and M. Wenzel. Eisbach: A proof method language for Isabelle. *J. Autom. Reasoning*, 56(3):261–282, 2016.
- [11] J. C. Blanchette et al. Hammering towards QED. *J. Formalized Reasoning*, 9(1):101–148, 2016.
- [12] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv:2009.03393*, 2020.
- [13] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented LMs. In *NeurIPS*, 2023.
- [14] S. Mac Lane and I. Moerdijk. *Sheaves in Geometry and Logic*. Springer, 1994.
- [15] M. Artin, A. Grothendieck, and J.-L. Verdier. *SGA 4*. Springer LNM 269/270/305, 1972–1973.
- [16] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality. *New J. Phys.*, 13:113036, 2011.
- [17] T. Ringer et al. Proof repair across type equivalences. In *PLDI*, ACM, 2021.
- [18] A. Solar-Lezama. Program synthesis by sketching. PhD thesis, UC Berkeley, 2008.
- [19] E. Torlak and R. Bodík. A lightweight symbolic virtual machine for solver-aided host languages. In *PLDI*, ACM, 2014.
- [20] A. Bundy. The use of explicit plans to guide inductive proofs. In *CADE*, Springer LNCS, 1988.
- [21] A. Bundy. *Rippling: Meta-level guidance for mathematical reasoning*. Cambridge University Press, 2005.
- [22] E. Visser. Program transformation with Stratego/XT. In *Domain-Specific Program Generation*, Springer LNCS, 2004.
- [23] P. Borovanský et al. ELAN from a rewriting logic point of view. *Theoretical Computer Science*, 285(2):155–185, 2002.
- [24] E. Melis. The heuristic solver: Analogy in proof search. In *Advances in Analogy Research*, New Bulgarian University, 1998.